Assignment-21.5

Name: Arjun Manoj

H. No:2303A52134

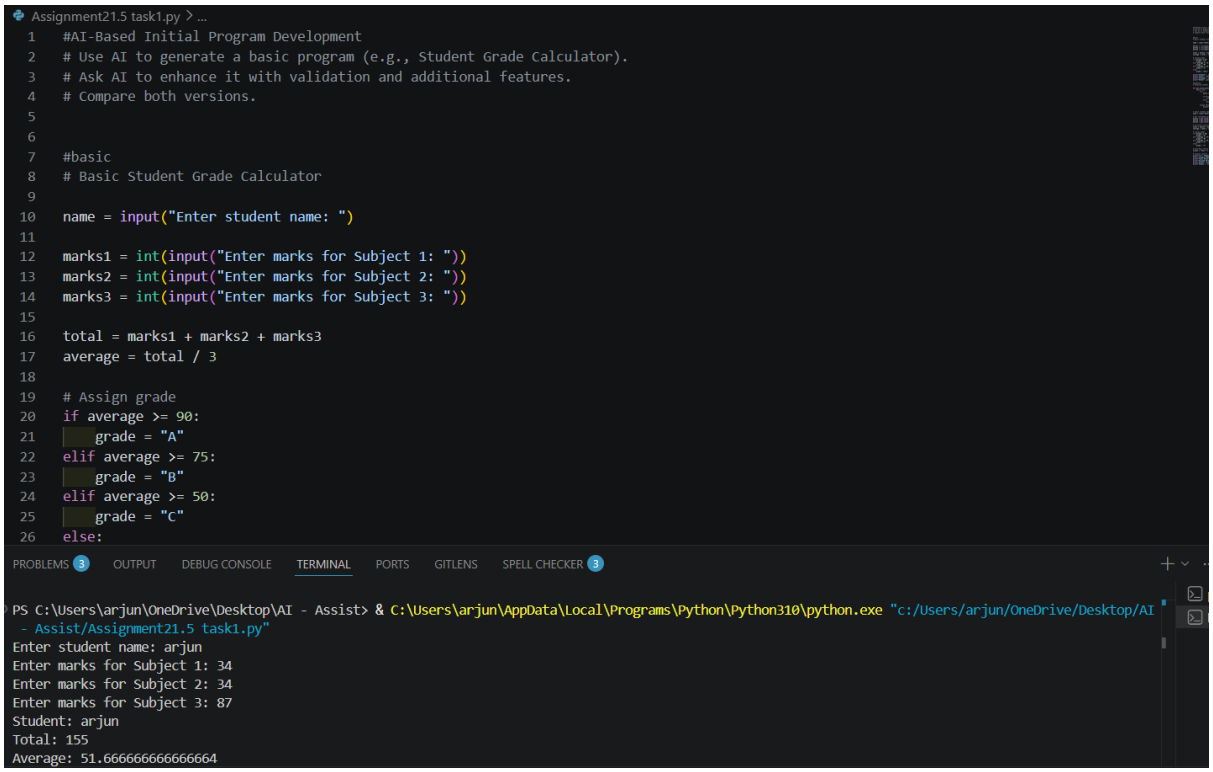
Batch:44

Task A – AI-Based Initial Program Development

Prompt:

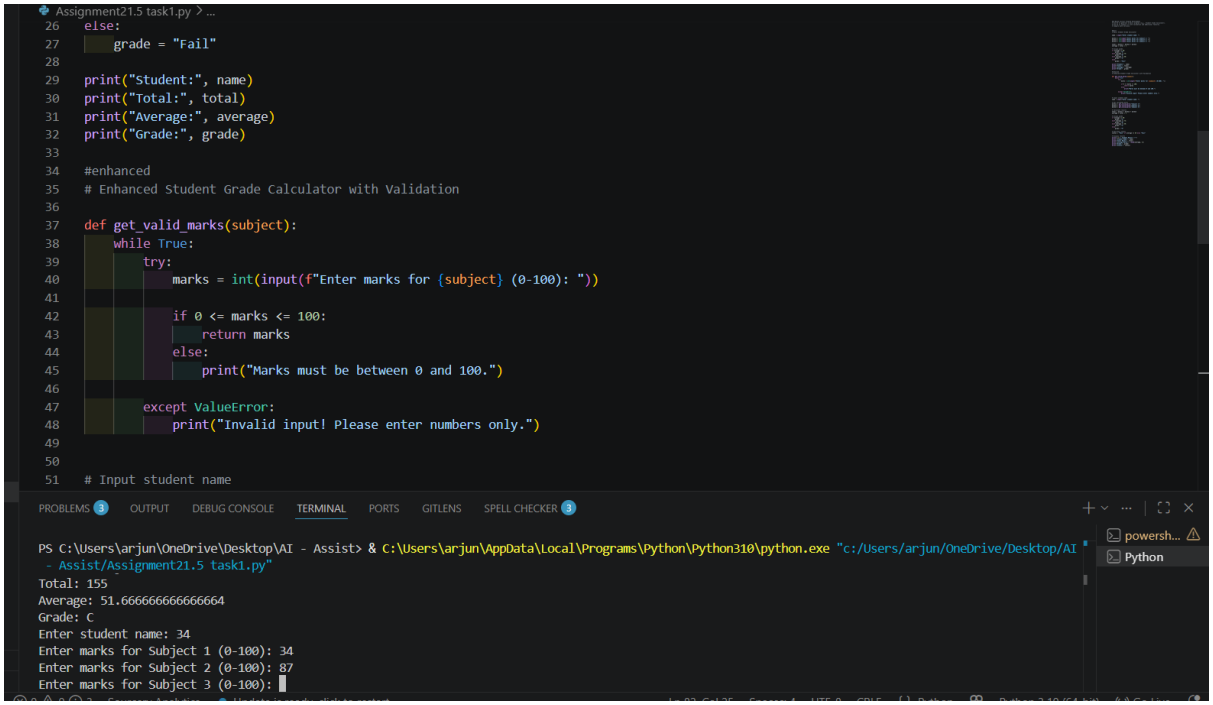
Generate a basic Python program that calculates the total and average marks of students and assigns grades. Then enhance the program by adding input validation, error handling, and additional features such as grade classification and pass/fail status. Compare both versions and analyze the improvements.

Code:

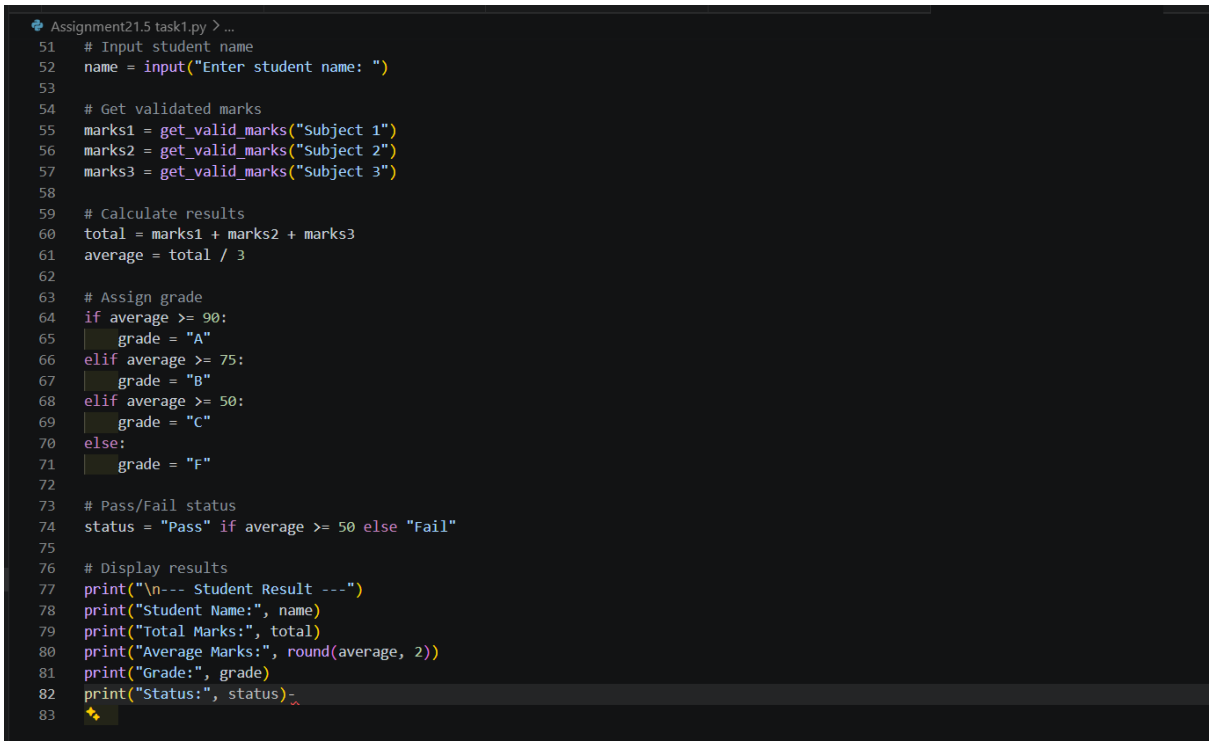


```
Assignment21.5 task1.py > ...
1   #AI-Based Initial Program Development
2   # Use AI to generate a basic program (e.g., Student Grade Calculator).
3   # Ask AI to enhance it with validation and additional features.
4   # Compare both versions.
5
6
7   #basic
8   # Basic Student Grade Calculator
9
10  name = input("Enter student name: ")
11
12  marks1 = int(input("Enter marks for Subject 1: "))
13  marks2 = int(input("Enter marks for Subject 2: "))
14  marks3 = int(input("Enter marks for Subject 3: "))
15
16  total = marks1 + marks2 + marks3
17  average = total / 3
18
19  # Assign grade
20  if average >= 90:
21      grade = "A"
22  elif average >= 75:
23      grade = "B"
24  elif average >= 50:
25      grade = "C"
26  else:
```

```
PROBLEMS 3  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS  SPELL CHECKER 3

PS C:\Users\arjun\OneDrive\Desktop\AI - Assist> & C:\Users\arjun\AppData\Local\Programs\Python\Python310\python.exe "c:/Users/arjun/OneDrive/Desktop/AI
 - Assist/Assignment21.5 task1.py"
Enter student name: arjun
Enter marks for Subject 1: 34
Enter marks for Subject 2: 34
Enter marks for Subject 3: 87
Student: arjun
Total: 155
Average: 51.666666666666664
```



```
Assignment21.5 task1.py > ...
26  else:
27      grade = "Fail"
28
29  print("Student:", name)
30  print("Total:", total)
31  print("Average:", average)
32  print("Grade:", grade)
33
34  #enhanced
35  # Enhanced Student Grade Calculator with Validation
36
37  def get_valid_marks(subject):
38      while True:
39          try:
40              marks = int(input(f"Enter marks for {subject} (0-100): "))
41
42              if 0 <= marks <= 100:
43                  return marks
44              else:
45                  print("Marks must be between 0 and 100.")
46
47          except ValueError:
48              print("Invalid input! Please enter numbers only.")
49
50
51  # Input student name

PROBLEMS 3  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS  SPELL CHECKER 3

PS C:\Users\arjun\OneDrive\Desktop\AI - Assist> & C:\Users\arjun\AppData\Local\Programs\Python\Python310\python.exe "c:/Users/arjun/OneDrive/Desktop/AI - Assist/Assignment21.5 task1.py"
Total: 155
Average: 51.666666666666664
Grade: C
Enter student name: 34
Enter marks for Subject 1 (0-100): 34
Enter marks for Subject 2 (0-100): 87
Enter marks for Subject 3 (0-100):

powersh...
Python
```



```
Assignment21.5 task1.py > ...
51  # Input student name
52  name = input("Enter student name: ")
53
54  # Get validated marks
55  marks1 = get_valid_marks("Subject 1")
56  marks2 = get_valid_marks("Subject 2")
57  marks3 = get_valid_marks("Subject 3")
58
59  # Calculate results
60  total = marks1 + marks2 + marks3
61  average = total / 3
62
63  # Assign grade
64  if average >= 90:
65      grade = "A"
66  elif average >= 75:
67      grade = "B"
68  elif average >= 50:
69      grade = "C"
70  else:
71      grade = "F"
72
73  # Pass/Fail status
74  status = "Pass" if average >= 50 else "Fail"
75
76  # Display results
77  print("\n--- Student Result ---")
78  print("Student Name:", name)
79  print("Total Marks:", total)
80  print("Average Marks:", round(average, 2))
81  print("Grade:", grade)
82  print("Status:", status)
83
```

Observation:

The initial AI-generated program successfully calculated total marks, average, and grade but lacked input validation and error handling, making it prone to crashes and incorrect data entry. The enhanced version improved reliability by adding input validation, range checking (0–100), and exception handling to prevent invalid inputs. Additional features such as pass/fail status and formatted output made the program

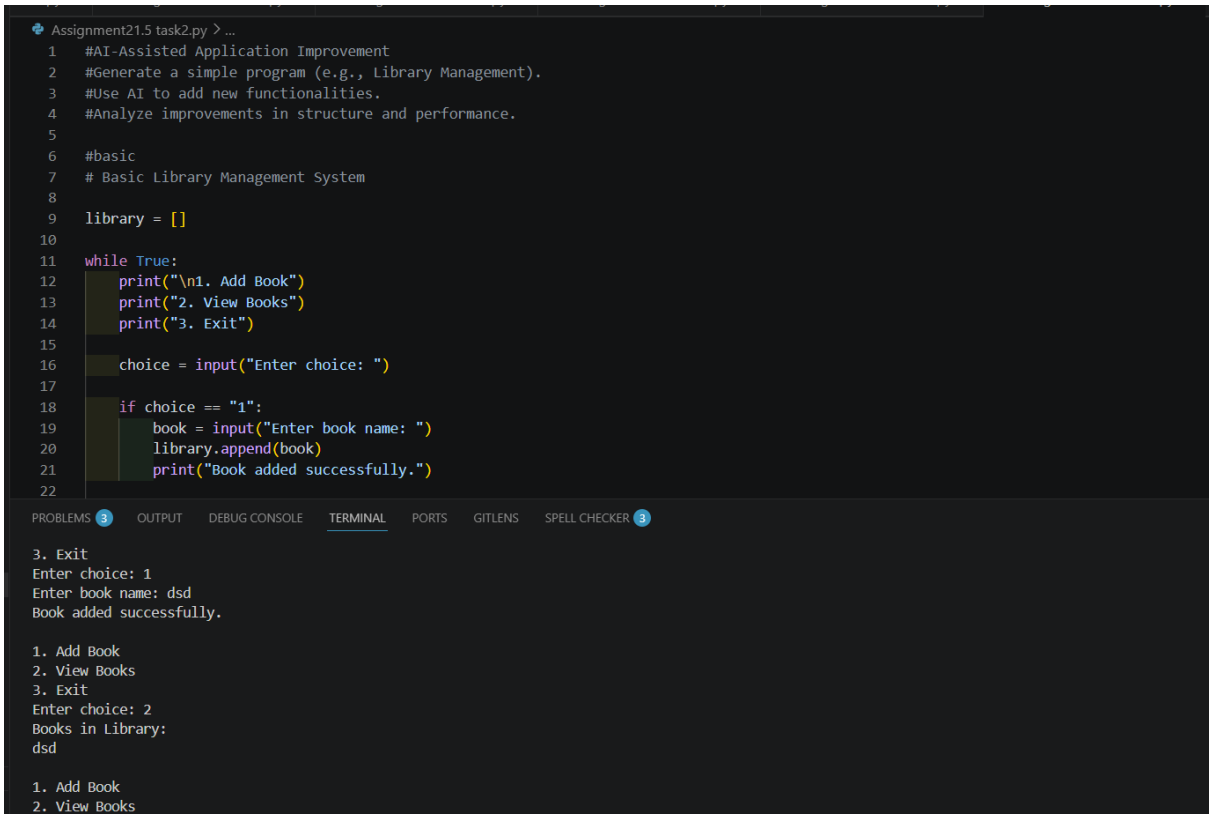
more user-friendly and robust. Overall, the enhanced version is safer, more reliable, and suitable for real-world usage.

Task B – AI-Assisted Application Improvement

Prompt:

Generate a simple Python Library Management program that allows adding and viewing books. Then enhance the program by adding new functionalities such as searching books, deleting books, and improving program structure using functions. Analyze how the improvements affect the structure and performance of the program.

Code:



Assignment21.5 task2.py > ...

```
    elif choice == "2":
        print("Books in Library:")
        for book in library:
            print(book)

    elif choice == "3":
        print("Exiting program.")
        break

    else:
        print("Invalid choice.")

#enhanced
# Enhanced Library Management System

library = []


# Function to add book
def add_book():
    book = input("Enter book name: ")
```

PROBLEMS 3 OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS SPELL CHECKER 3

```
Enter book name: dsd
Book added successfully.

1. Add Book
2. View Books
3. Exit
Enter choice: 2
Books in Library:
dsd

1. Add Book
2. View Books
3. Exit
Enter choice:
```

Assignment21.5 task2.py > ...

```
def view_books():
    if not library:
        print("Library is empty.")
    else:
        print("\nBooks in Library:")
        for index, book in enumerate(library, start=1):
            print(index, ".", book)


# Function to search book
def search_book():
    book = input("Enter book name to search: ")

    if book in library:
        print("Book found in library.")
    else:
        print("Book not found.")


# Function to delete book
def delete_book():
    book = input("Enter book name to delete: ")

    if book in library:
        library.remove(book)
        print("Book deleted successfully.")
    else:
        print("Book not found.")


# Main menu loop
while True:
    print("\n===== Library Menu =====")
    print("1. Add Book")
    print("2. View Books")
    print("3. Search Book")
    print("4. Delete Book")
```

```python

# Main menu loop
while True:
    print("\n===== Library Menu =====")
    print("1. Add Book")
    print("2. View Books")
    print("3. Search Book")
    print("4. Delete Book")
    print("5. Exit")

    choice = input("Enter choice: ")

    if choice == "1":
        add_book()

    elif choice == "2":
        view_books()

    elif choice == "3":
        search_book()

    elif choice == "4":
        delete_book()

    elif choice == "5":
        print("Exiting program.")
        break

    else:
        print("Invalid choice.")
```

Observation:

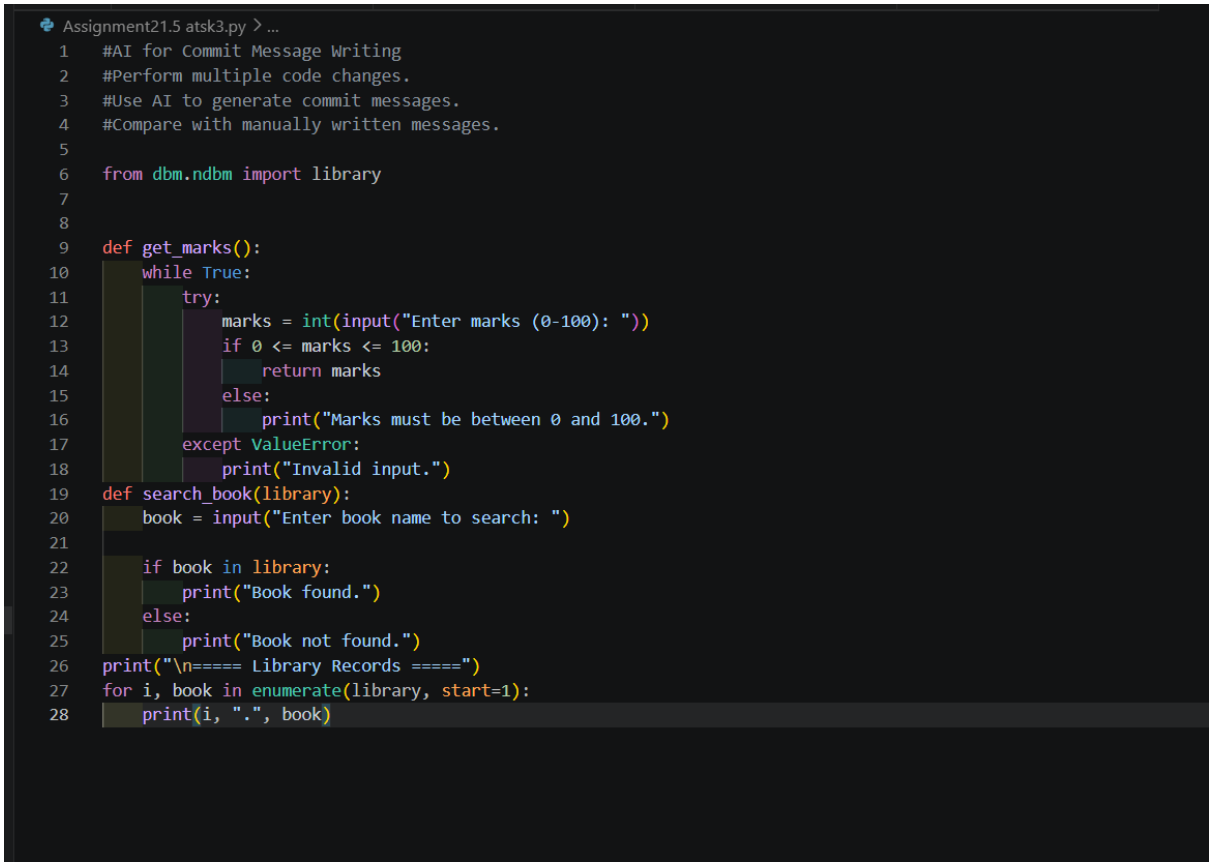
The initial AI-generated Library Management program supported only basic operations such as adding and viewing books, but lacked advanced features and proper structure. The improved version introduced new functionalities like searching and deleting books, making the system more useful and interactive. Functions were added to organize the code into modular sections, improving readability, maintainability, and scalability. Overall, the enhanced program demonstrates better structure, improved performance efficiency, and easier expansion for future features such as file storage or database integration.

Task C – AI for Commit Message Writing

Prompt:

Perform multiple code changes in a Python project such as adding validation, fixing errors, and improving features. Use AI to generate meaningful commit messages for each change. Compare the AI-generated commit messages with manually written commit messages and analyze their clarity and usefulness.

Code:



Observation:

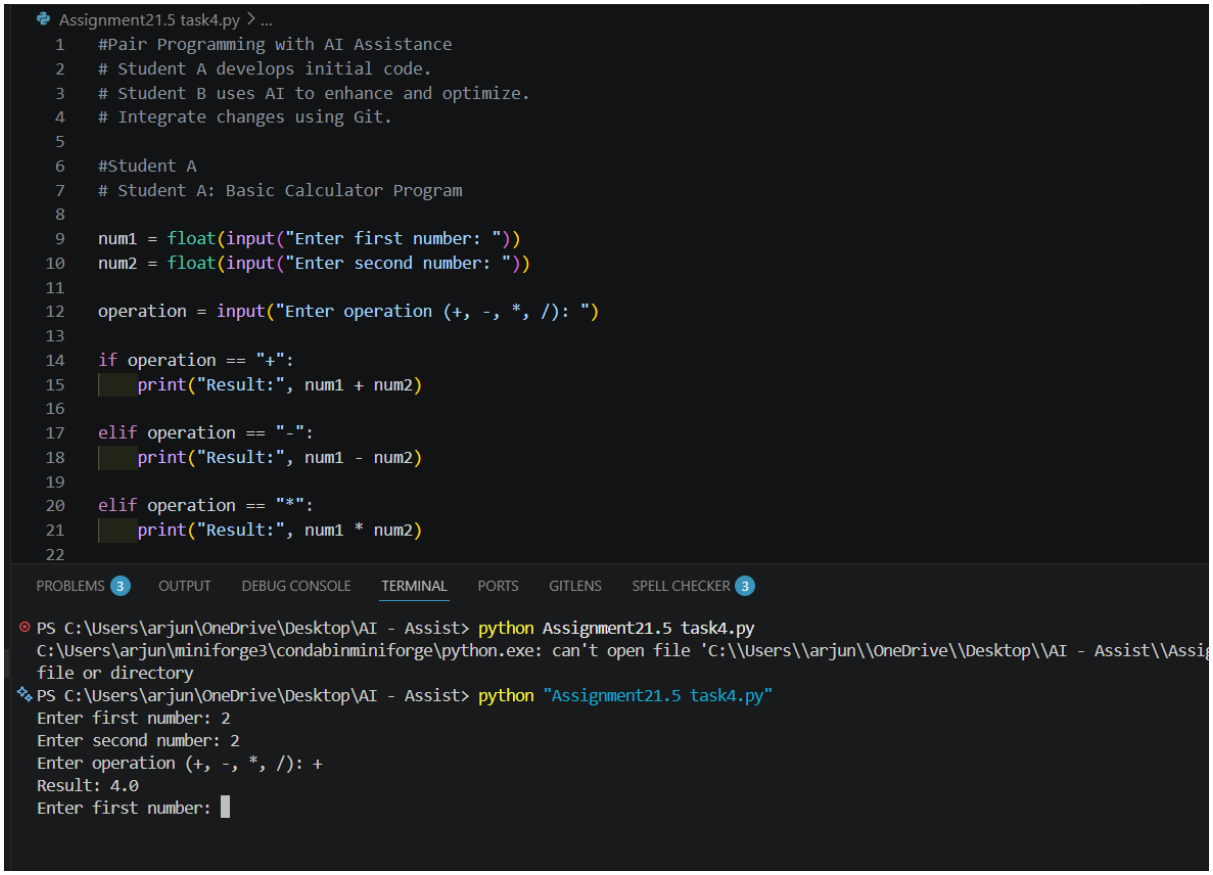
AI-generated commit messages were more descriptive and clearly explained the purpose of each code change, making them easier to understand during project tracking. Manually written messages were shorter but less informative, which could make it harder to identify specific changes later. Overall, AI-assisted commit message writing improves clarity, consistency, and maintainability in version control workflows.

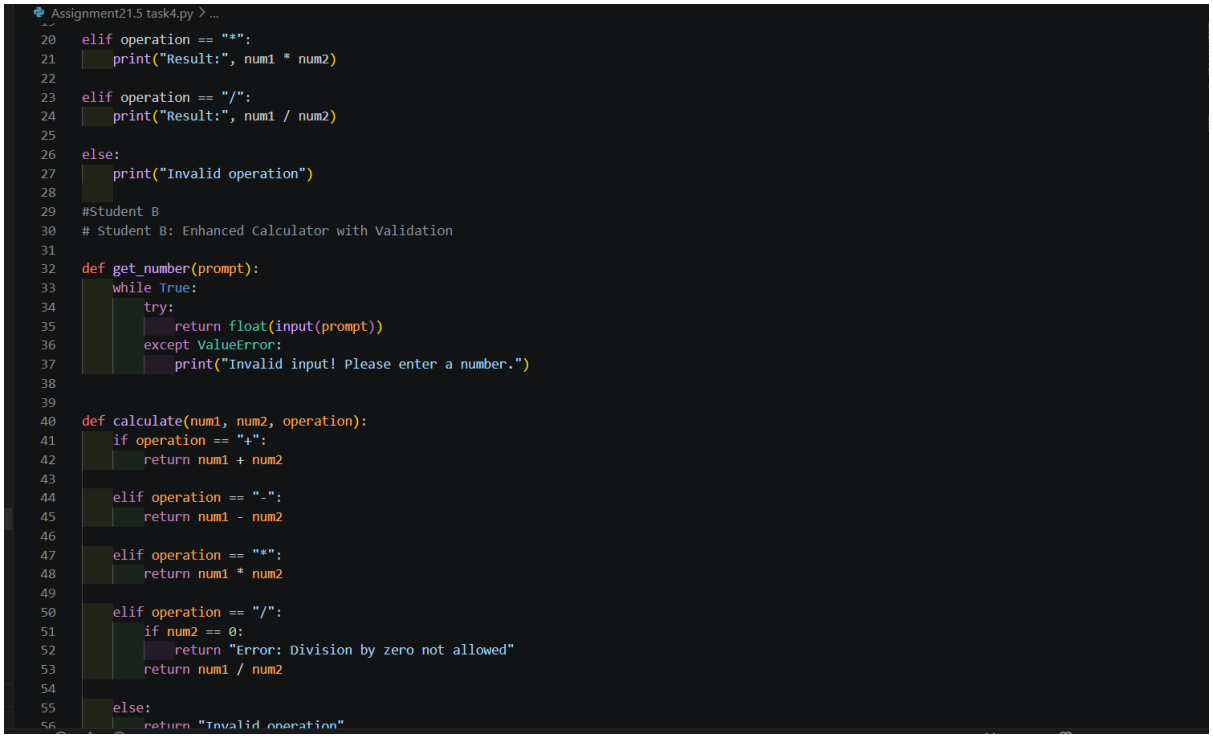
Task D – Pair Programming with AI Assistance

Prompt:

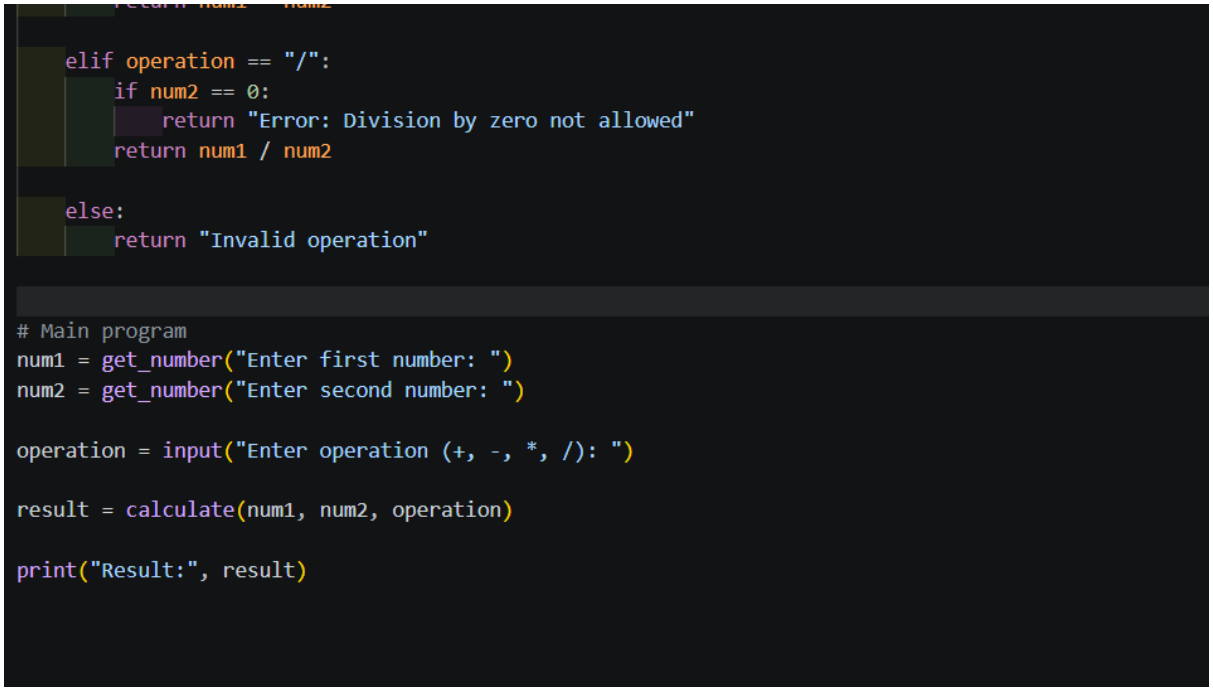
Student A develops an initial Python program (such as a basic calculator or library system). Student B uses AI assistance to enhance and optimize the program by adding validation, improving structure, and adding new features. Integrate the improved code using Git by committing and merging the changes. Analyze how AI-assisted pair programming improves code quality and teamwork.

Code:





```
Assignment21.5 task4.py > ...
    elif operation == "*":
        print("Result:", num1 * num2)

    elif operation == "/":
        print("Result:", num1 / num2)

    else:
        print("Invalid operation")

    #Student B
    # Student B: Enhanced Calculator with Validation

    def get_number(prompt):
        while True:
            try:
                return float(input(prompt))
            except ValueError:
                print("Invalid input! Please enter a number.")


    def calculate(num1, num2, operation):
        if operation == "+":
            return num1 + num2

        elif operation == "-":
            return num1 - num2

        elif operation == "*":
            return num1 * num2

        elif operation == "/":
            if num2 == 0:
                return "Error: Division by zero not allowed"
            return num1 / num2

        else:
            return "Invalid operation"
```



```
    elif operation == "/":
        if num2 == 0:
            return "Error: Division by zero not allowed"
        return num1 / num2

    else:
        return "Invalid operation"


# Main program
num1 = get_number("Enter first number: ")
num2 = get_number("Enter second number: ")

operation = input("Enter operation (+, -, *, /): ")

result = calculate(num1, num2, operation)

print("Result:", result)
```

Observation:

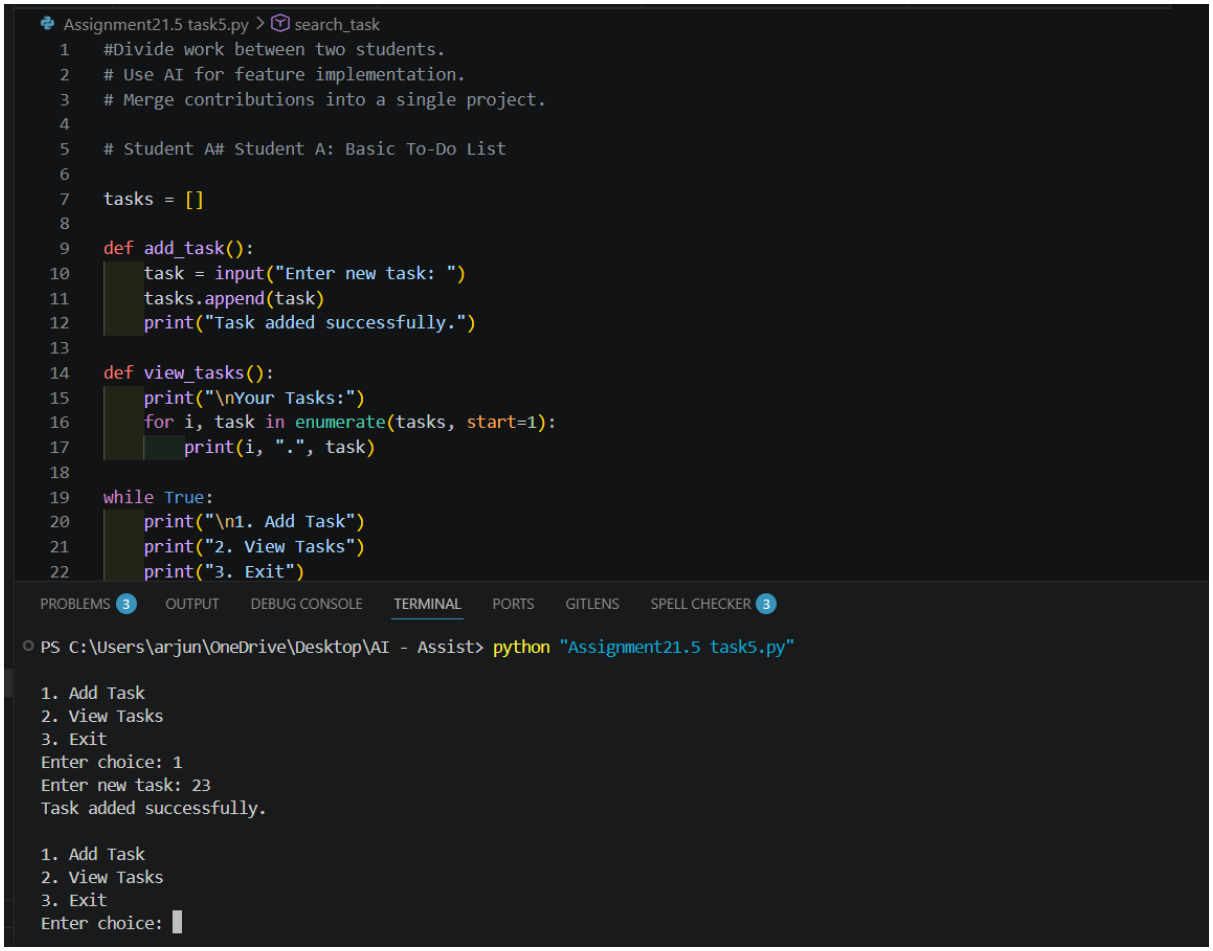
Student A created a basic working program, while Student B used AI assistance to enhance the code with input validation, modular functions, and error handling. Using Git allowed both versions to be managed efficiently through branching and merging. AI-assisted pair programming improved code quality, reduced errors, and encouraged better collaboration and structured development.

Task E – Collaborative Development using Branches

Prompt:

Divide the development of a Python project between two students using Git branches. Student A implements the basic functionality, while Student B uses AI assistance to add new features. Each student works on a separate branch, and the contributions are merged into a single final project. Analyze how branching supports collaborative development and feature integration.

Code:



```python
while True:
    print("\n1. Add Task")
    print("2. View Tasks")
    print("3. Exit")

    choice = input("Enter choice: ")

    if choice == "1":
        add_task()

    elif choice == "2":
        view_tasks()

    elif choice == "3":
        print("Exiting program.")
        break

    else:
        print("Invalid choice.")

# Student B: Enhanced To-Do List with Priorities
# Student B: Additional Features

def delete_task(tasks):
    view_tasks(tasks)

    task_no = int(input("Enter task number to delete: "))

    if 1 <= task_no <= len(tasks):
        removed = tasks.pop(task_no - 1)
        print("Deleted task:", removed)
    else:
        print("Invalid task number.")


def search_task(tasks):
    keyword = input("Enter keyword to search: ")
```

```python


def search_task(tasks):
    keyword = input("Enter keyword to search: ")

    found = False

    for task in tasks:
        if keyword.lower() in task.lower():
            print("Found:", task)
            found = True

    if not found:
        print("No matching task found.")
```

Observation:

Dividing work into separate branches allowed Student A and Student B to develop features independently without affecting the main program. AI assistance helped Student B quickly implement additional features such as delete and search functions. Merging branches combined both contributions into a single project, improving collaboration, reducing

conflicts, and making the development process more organized and efficient.